

Theory Section: Kernel Methods and Neural Network Feature Representations

1 Theory

1.1 Kernel Methods

1.1.1 Valid Kernel Function

A kernel function is a function that computes the similarity between two inputs in a (possibly high-dimensional) feature space without explicitly computing the mapping to that space.

Formally, a function $K(x, z)$ is a valid kernel if there exists a feature mapping $\phi(x)$ to some feature space $\mathcal{H}$ such that:

$$K(x, z) = \phi(x)^T \phi(z)$$

This is called the **kernel trick**, allowing algorithms (like Support Vector Machines) to operate in high-dimensional spaces efficiently.

Properties of a valid kernel

1. Symmetry

$$K(x, z) = K(z, x)$$

2. Positive Semi-Definite (PSD)

For any set of points $x_1, x_2, \dots, x_n$, the kernel matrix

$$K_{ij} = K(x_i, x_j)$$

must satisfy

$$\sum_{i=1}^n \sum_{j=1}^n c_i c_j K(x_i, x_j) \geq 0$$

for all real coefficients $c_1, c_2, \dots, c_n$.

This means the kernel matrix must be positive semi-definite.

Examples of valid kernels

- **Linear kernel**

$$K(x, z) = x^T z$$

- **Polynomial kernel**

$$K(x, z) = (x^T z + c)^d$$

- **RBF / Gaussian kernel**

$$K(x, z) = \exp(-\gamma \|x - z\|^2)$$

1.1.2 Mercer's Theorem

Mercer's Theorem provides the mathematical condition under which a function can be used as a kernel.

Statement

A continuous symmetric function $K(x, z)$ defined on a compact domain is a valid kernel if and only if the kernel matrix generated from it is positive semi-definite for any finite set of data points.

Equivalently, Mercer's theorem states that such a kernel can be expressed as an expansion:

$$K(x, z) = \sum_{i=1}^{\infty} \lambda_i \psi_i(x) \psi_i(z)$$

where:

- $\lambda_i \geq 0$ are non-negative eigenvalues
- $\psi_i(x)$ are orthogonal eigenfunctions

This shows that the kernel corresponds to an inner product in an infinite-dimensional feature space, which justifies the kernel trick used in algorithms like Support Vector Machines.

1.2 Neural Network Hidden Layer as a Learned Feature Representation $\phi(x)$

In machine learning, a feature representation $\phi(x)$ transforms the original input x into a new feature space where learning becomes easier (for example, linear separation).

1.2.1 Hidden Layer as Feature Mapping

In a neural network, each hidden layer performs a transformation of the input.

For an input x , a hidden layer computes:

$$h = \sigma(Wx + b)$$

where:

- W = weight matrix
- b = bias vector
- $\sigma(\cdot)$ = activation function (ReLU, sigmoid, etc.)
- h = hidden layer output

This hidden representation can be interpreted as a feature mapping:

$$\phi(x) = h = \sigma(Wx + b)$$

Thus, the hidden layer learns a transformation of the input data into a new feature space where the final layer (often linear) can perform classification or regression.

Key Idea

The neural network automatically learns useful features from data during training.

1.2.2 Example Intuition

Input x	Hidden Layer Representation $\phi(x)$	Purpose
Raw pixels	Edges, shapes	Vision tasks
Text tokens	Semantic embeddings	NLP
Tabular features	Nonlinear combinations	Prediction

1.2.3 Relation to the Kernel Trick

Kernel methods (like Support Vector Machines) also use a feature mapping $\phi(x)$.

A kernel function is defined as:

$$K(x, z) = \phi(x)^T \phi(z)$$

Instead of explicitly computing $\phi(x)$, kernel methods compute the inner product in feature space directly using the kernel function.

1.2.4 Key Difference

Neural Networks	Kernel Methods
Learn $\phi(x)$ from data	Use a predefined $\phi(x)$ via kernel
Feature map is explicit	Feature map is implicit
Parameters learned through training	Kernel function fixed beforehand

1.2.5 Conceptual Connection

Both approaches aim to map inputs into a higher-dimensional feature space where the problem becomes easier (often linearly separable).

Neural Networks

Learn the mapping:

$$\phi(x) = \sigma(Wx + b)$$

Kernel Methods

Use a kernel function to compute:

$$K(x, z) = \phi(x)^T \phi(z)$$

without explicitly computing $\phi(x)$.

2 analysis of 7th question

2.1 Kernel Methods

Kernel methods allow machine learning algorithms to operate in high-dimensional feature spaces without explicitly computing the transformation. Instead of computing a mapping $\phi(x)$ directly, the algorithm evaluates an inner product between transformed features using a kernel function.

A valid kernel function must satisfy the property that it can be written as an inner product in some feature space:

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

This enables the **kernel trick**, where algorithms such as Support Vector Machines (SVMs) can work in very high dimensional spaces efficiently.

Mercer's Theorem states that a continuous symmetric function $K(x, z)$ is a valid kernel if and only if the corresponding kernel matrix is positive semi-definite for any dataset. This guarantees that the kernel corresponds to a legitimate inner product in some feature space.

Thus, kernel methods transform nonlinear problems in the original space into linear problems in a higher-dimensional feature space.

3 Neural Feature Representations

3.1 Neural Network as Feature Extractor

In this part, a multi-layer perceptron (MLP) was trained on the dataset to predict the mess duration. The architecture used multiple hidden layers with ReLU activations and a sigmoid output layer.

After training the network, the penultimate hidden layer was used as a learned feature representation.

$$\phi_{NN}(x) = h^{(L-1)}(x)$$

These neural features represent higher-level abstractions learned by the neural network.

Instead of using the raw input features, the neural network learns representations that capture patterns in the data.

To analyze these learned representations, t-SNE dimensionality reduction was applied to the penultimate layer outputs. The t-SNE visualization showed clusters of points corresponding to different values of the target variable. This indicates that the neural network successfully learned meaningful feature embeddings.

The clustering structure in the t-SNE plot demonstrates that samples with similar target values are mapped close together in the learned feature space.

Thus, the neural network acts as an automatic feature extractor, transforming the input data into a representation that makes prediction easier.

4 Comparison with Standard Kernels

In this section, Support Vector Regression (SVR) was implemented using several kernel functions. These kernels measure similarity between samples in different ways.

4.1 Linear Kernel

$$k(x_i, x_j) = x_i^T x_j$$

The linear kernel directly computes the dot product of two samples. This assumes that the relationship between input features and the target variable is linear.

Since the dataset likely contains nonlinear relationships, the linear kernel performed poorly.

Test MSE obtained:

Linear Kernel = 29.82

4.2 Polynomial Kernel

$$k(x_i, x_j) = (x_i^T x_j + c)^d$$

Polynomial kernels allow the model to capture nonlinear relationships by mapping the data into higher-degree polynomial feature spaces.

Results

Degree 2 $\rightarrow$ MSE = 24.09

Degree 3 $\rightarrow$ MSE = 14.28

Increasing the polynomial degree improved performance because the model gained the ability to represent more complex patterns in the data.

4.3 RBF (Gaussian) Kernel

$$k(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

The RBF kernel measures similarity based on the Euclidean distance between samples. It is widely used because it can model highly nonlinear relationships.

Results

$\gamma = 0.01 \rightarrow \text{MSE} = 22.91$

$\gamma = 0.1 \rightarrow \text{MSE} = 15.11$

$\gamma = 1.0 \rightarrow \text{MSE} = 28.06$

The best performance occurred when $\gamma = 0.1$, which provides a good balance between smoothness and flexibility.

A very large γ value makes the kernel too localized, which can lead to overfitting.

4.4 Neural Kernel

The neural kernel uses features extracted from the trained neural network.

$$k(x_i, x_j) = \phi_{NN}(x_i)^T \phi_{NN}(x_j)$$

Instead of computing similarity in the original feature space, the kernel operates in the learned neural feature space.

Results

Neural Kernel MSE = 6.28

This is significantly better than all other kernels.

The neural network transforms the data into a feature space where samples with similar target values are closer together. As a result, the kernel regression model can perform more accurate predictions.

5 Overall Comparison

Kernel	Test MSE
Linear	29.82
Polynomial (d=2)	24.09
Polynomial (d=3)	14.28
RBF ($\gamma = 0.01$)	22.91
RBF ($\gamma = 0.1$)	15.11
RBF ($\gamma = 1.0$)	28.06
Neural Kernel	6.28

6 Key Insights

- Linear kernels are insufficient for modeling complex relationships in the dataset.
- Polynomial kernels improve performance by introducing nonlinear feature transformations.

- RBF kernels capture nonlinear relationships effectively but require careful parameter tuning.
- Neural kernels perform best because the neural network learns task-specific feature representations before applying the kernel method.